

## K-Nearest Neighbor Classification

An algorithm **used for classification**, is a **lazy-learning algorithm**

### The idea

**Examples** are **vectors of features**

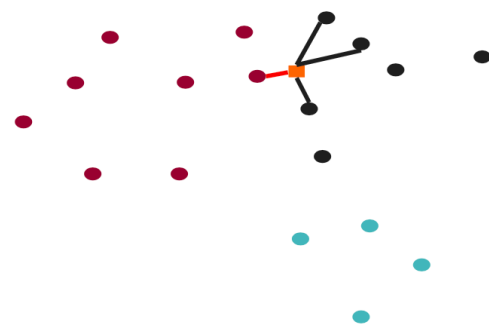
- **Training examples** can be represented as **points in a  $d$ -dimensional space**
  - $d$  is the number of features

**To classify** a given **unlabeled** example  $d$

- We **find the  $k$  nearest neighbors** of  $d$
- **As label** we choose the **majority label within the  $k$  nearest neighbors**



Training examples,  
each color is a category



Classification of an  
unlabelled testing example

### Measuring the “nearest”

We need to distinguish 2 key concepts:

- **Distance** → **the higher** it is, **the more different** two data objects are
- **Similarity** → **the higher** it is, **the more similar** two data objects are

### Minkowski distance

Given two points  $a$  and  $b$  their **distance** is

$$D(a, b) = \left( \sum_{i=1}^d |a_i - b_i|^r \right)^{1/r}$$

Where:

- $r$  is a parameter
  - $r = 1$ : city block distance
  - $r = 2$ : **Euclidian** distance
  - $r \rightarrow \infty$ : supremum distance
- $d$  is the number of dimensions

⚠ Sometimes **features are not comparable** → use of **standardization** and **scaling**

### Standardization or Z-score normalization

**Rescales** the data so that:

- The **mean is 0**
- The **standard deviation from the mean is 1**

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

Where:

- $\mu$  is mean
- $\sigma$  is standard deviation from the mean

### Min-Max Scaling

**Scales data to a fixed range**

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

### Cosine similarity

Given two vectors  $\vec{a}$  and  $\vec{b}$  **the cosine of the angle between them is:**

$$C(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

**Focuses on the direction** of vectors:

- $C(\vec{a}, \vec{b}) = 1$ : **aligned** → **maximum similarity**
- $C(\vec{a}, \vec{b}) = 0$ : **orthogonal** → **no similarity**
- $C(\vec{a}, \vec{b}) = -1$ : **opposite**

## Decision boundaries

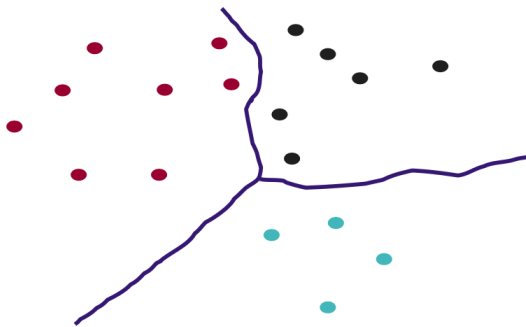
**Places** in the features space that **define where the classification of an example changes**

- Are a **subset of the Voronoi diagram**

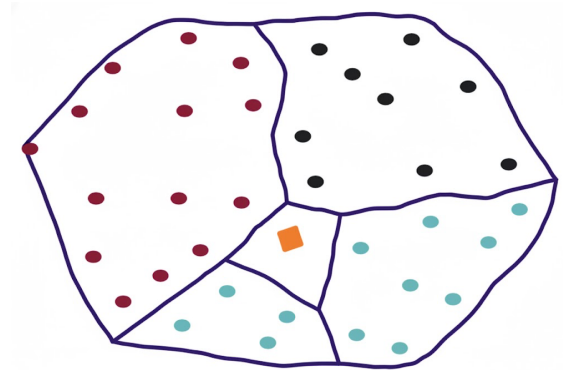
## Voronoi diagram

Describes **the area** that are **nearest to any given point**

- **Each line is equally distant** from two points



Decision boundaries



Voronoi diagram

## Choosing $k$

The **value of  $k$**  defines the **level of overfitting and underfitting**

How to choose  $k$ ?

- Common **heuristics**
  - $k = 3, 5, 7$
  - $k$  should be an **odd number**
  - $k < \sqrt{N}$  (where  $N$  is the number of training examples)
- **Validation set** and **cross-validation techniques** can be used

## Cross-validation

Used to **estimate generalization performance** and **select a model's hyperparameters**

- Split the dataset into  $m$  equal folds
- For each fold
  - Train the models on  $m - 1$  folds
  - Validate the model on the remaining fold
  - Repeat  $m$  times, using each fold once for validation
- Compute the average validation error across folds

## Weighted K-NN

**Closer neighbors** have a **larger weight**

- Their influence is higher

A common functions that describes a point weight is:

$$w_i = \frac{1}{d_i + \varepsilon}$$

## Curse of dimensionality

The **data space grows exponentially with the number of dimensions**

- → **The size of the dataset must also grow exponentially**
  - **To keep** the same **density**
  - **Without** increase in the size of the dataset, **K-NN loses performance**

## Computational costs

**Training time:**  $O(1)$

- K-NN is a lazy-learning algorithm

**Prediction time:**  $O(dN)$

- Compute distance for all  $N$  datapoints:  $O(dN)$
- Order the distances and find the minimum  $k$

**Space complexity:**  $O(dN)$

- The whole dataset must be kept on RAM memory